МАРШРУТ 01

Системный анализ с нуля до Junior

19 блоков · сквозной проект · четыре контрольные точки

Этот гид рассчитан на человека, который ещё не работал системным аналитиком. Здесь можно начать без программирования и двигаться по одному проекту: сначала разобраться в задаче заказчика, затем описать поведение системы, данные, интерфейсы и проверки. На выходе у тебя будет комплект связанных работ, который можно показать ментору и использовать в портфолио.

Цель маршрута - подготовить тебя к ограниченной аналитической задаче под ревью более опытного коллеги. Например, описать запись на занятие, добавить фильтр в список или согласовать поля нового метода API. Название Junior в разных компаниях означает разное; здесь уровень определяется тем, какую работу ты можешь сделать и объяснить.

Содержание

Как проходить маршрут 3
Сквозной проект Сервис записи на занятия 5
J00 Подготовь рабочее место 6
J01 Разберись в работе аналитика и команды 7
J02 Пойми как запрос проходит через систему 8
J03 Выясни задачу заказчика и границы решения 9
J04 Научись писать проверяемые требования 10
J05 Опиши пользовательские сценарии 11
J06 Нарисуй бизнес процесс в BPMN 12
J07 Покажи взаимодействия и состояния в UML 13
J08 Спроектируй данные 14
J09 Освой SQL на одной таблице 15
J10 Соедини таблицы и проверь качество данных 16
J11 Разберись в форматах сообщений 17
J12 Спроектируй API первой версии 18
J13 Оформи OpenAPI и проверь запросы в Postman 19
J14 Опиши интеграцию с внешней системой 20
J15 Опиши интерфейс и доступ к данным 21
J16 Собери постановку и научись менять требования 22
J17 Проверь результат и помоги команде разобраться с ошибкой 23
J18 Защити проект и подготовь портфолио 24
Приложение А Шаблон постановки одной фичи 25
Приложение Б Вопросы с ориентирами для ответа 26
Приложение В Как использовать ИИ в обучении 27
Приложение Г Куда идти после Junior 28
Реестр источников этого маршрута 29

Как проходить маршрут

Внутри каждого блока есть обязательное чтение, дополнительные материалы, задание и самопроверка. Начни с обязательных источников в указанном порядке. Дополнительные открывай, если основной материал не помог или хочется разобраться глубже. Не нужно прочитать весь интернет перед первой практикой.

Рабочий цикл такой: прочитай небольшую часть, попробуй применить её в проекте, выпиши вопросы, закончи работу, объясни решение вслух и отправь на проверку. После замечаний обнови ту же работу. Отмечай не «посмотрел видео», а «могу сделать без подсказки».

Для английских источников разрешён перевод страницы. Сохраняй английские названия полей и терминов: они встретятся в документации и на работе. Первое чтение стандарта не должно превращаться в попытку выучить его целиком: в заданиях указан нужный фрагмент.

Оценки времени - наш ориентир на учебную работу, а не статистика курсов. В сумме блоки занимают около 350 часов; с повторением и доработками разумно планировать 400-460 часов. При 12 часах в неделю это примерно 8-9 месяцев до завершения учебной части. При 20 часах - около 5-6 месяцев. Поиск работы и паузы считаются отдельно. После первых двух недель пересчитай план по своему темпу.

Навигация по блокам

Шаг	Что изучаем	Ориентир	Что сдаём
J00	Подготовка рабочего места	6 ч	Папка проекта и журнал обучения
J01	Профессия и разработка ПО	10 ч	Карта ролей и путь задачи
J02	Интернет и HTTP	14 ч	Разбор настоящего запроса
J03	Задача заказчика и границы	16 ч	Vision, вопросы и глоссарий
J04	Требования и их качество	20 ч	Каталог требований
J05	User Story и Use Case	18 ч	Истории, сценарии и приёмка
J06	Процессы в BPMN	18 ч	AS IS и TO BE
J07	UML и жизненный цикл	18 ч	Sequence и таблица переходов
J08	Данные и ER модель	22 ч	ERD и словарь данных
J09	SQL с одной таблицей	24 ч	Запросы и ожидаемые результаты
J10	SQL с несколькими таблицами	24 ч	Отчёты и диагностика данных
J11	JSON, XML и схемы	14 ч	Примеры сообщений и ограничения
J12	Проектирование REST API	24 ч	Контракт методов
J13	OpenAPI и Postman	24 ч	Спецификация и коллекция
J14	Интеграции и webhooks	18 ч	Интеграционная спецификация
J15	Интерфейс и права доступа	16 ч	Состояния экранов и матрица прав
J16	Постановка и изменения	18 ч	Полная постановка одной фичи
J17	Проверка и сопровождение	16 ч	Приёмка, дефекты и разбор логов
J18	Защита и выход на рынок	28 ч	Защищённый проект и портфолио

J09 и J10 можно тренировать понемногу параллельно с последующими блоками. J12 начинай после J02, J05, J08 и J11. Для J14 сначала закончи J07 и J12. Не пропускай требования ради SQL и API: без них техническое решение легко отвечает не на тот вопрос.

Как понять что блок пройден

Для каждой работы ментор ставит оценку от 0 до 3 по пяти признакам: верность бизнес-смыслу, полнота сценариев, техническая согласованность, возможность проверки, ясность объяснения. 0 - работа отсутствует или смысл нарушен; 1 - сделано с существенной помощью; 2 - решение самостоятельное, нужны локальные правки; 3 - решение устойчиво к новым условиям и аргументировано.

Для перехода дальше нужны минимум 2 по каждому признаку и исправленные критичные замечания. Критичное замечание - например, возможность получить чужую запись, потеря подтверждённого бронирования, противоречие между статусами в API и базе. Красивое оформление не компенсирует такую ошибку. При самостоятельном прохождении попроси практикующего аналитика проверить хотя бы контрольные точки после J05, J10, J14 и J18.

Сквозной проект Сервис записи на занятия

Ты работаешь аналитиком небольшого учебного центра. Сейчас администратор принимает заявки в мессенджере, отмечает места в таблице и вручную пишет клиентам. Иногда два человека получают подтверждение на последнее место, а отмены теряются. Центр хочет личный кабинет с расписанием и записью.

Первая версия: клиент смотрит доступные занятия, записывается, получает результат и отменяет запись; администратор создаёт занятия и видит список записавшихся. Внешний сервис уведомлений сообщает об успешной доставке сообщения или ошибке. Онлайн-оплаты, абонементы, лист ожидания и интеграция с государственными системами в эту версию не входят.

Учебные правила, которые нужно подтвердить на встрече с заказчиком: на занятии есть ограничение мест; у клиента не бывает двух активных записей на одно занятие; отмена доступна не позднее чем за 12 часов до начала; время центра - Europe/Moscow; клиент видит только свои записи. Эти правила придуманы для кейса. Если на учебной встрече заказчик их меняет, фиксируй решение и обновляй связанные работы.

Роли: клиент, администратор, внешний сервис уведомлений. Основные сущности: клиент, занятие, запись, попытка уведомления. Статус доставки сообщения и статус записи - разные вещи: неудачное SMS не должно автоматически отменять занятую клиентом позицию.

Храни работы по разделам: контекст, требования, процессы, модели, данные, API, интеграции, приёмка, решения. Веди таблицу вопросов с колонками «вопрос», «кому задан», «ответ», «дата», «что изменилось». Любое непроверенное предположение помечай явно.

J00 Подготовь рабочее место

Результат: ты умеешь открыть и изменить материалы проекта, отправить работу на ревью и найти предыдущую версию. Ориентир - 6 часов.

Понадобятся текстовый редактор с Markdown, diagrams.net или другой редактор диаграмм, браузер с DevTools, PostgreSQL, DBeaver Community и Postman. Базу и Postman можно установить ближе к J09 и J13. Не трать несколько дней на идеальный набор программ: достаточно одного инструмента для каждого вида работы.

Разберись с расширениями `.md`, `.json`, `.yaml`, `.sql`, `.drawio`, `.pdf`. Markdown хранит текст и простую разметку; JSON - данные; YAML удобен для спецификаций; SQL - команды для базы. PDF подходит для чтения, но изменяемые схемы и исходники сохраняй отдельно.

Обязательно:

- S67 Pro Git · RU. Прочитай разделы о системе контроля версий, создании репозитория и записи изменений. Пока хватит add, commit, status и просмотра diff.
- S30 PostgreSQL Downloads · EN. Выбери установщик под свою ОС. Для занятий используем PostgreSQL 17; более новая установленная версия допустима, если упражнения выполняются так же.
- S31 DBeaver Community · EN. Установи Community Edition и найди создание подключения.

Сделай: создай структуру проекта и README с целью обучения, текущим блоком и ближайшей работой. Добавь журнал с датой, затраченным временем, результатом и вопросами. Сохрани первую версию, измени одно требование и посмотри разницу версий. Подготовь один файл с диаграммой и экспорт в PDF.

Самопроверка: можешь ли ты восстановить вчерашний текст? Поймёт ли другой человек, где лежит последняя версия? Что хранится в исходнике схемы, а что в её картинке? Как передать работу без пересылки десяти файлов с названиями «финал2»?

Готово, когда: ментор открывает README, находит работы и оставляет комментарии без твоего устного сопровождения. Не публикуй рабочие документы работодателя: учебные данные достаточно придумать самостоятельно.

J01 Разберись в работе аналитика и команды

Результат: ты понимаешь, кому и для чего готовишь требования. Ориентир - 10 часов.

Бизнес-аналитик чаще выясняет потребность, процесс и ограничения бизнеса; системный аналитик детализирует поведение информационной системы, данные и взаимодействия. В небольшой команде это может делать один человек. Product Owner определяет ценность и порядок работ, разработчики реализуют решение, QA проверяет его. Архитектор помогает с решениями на уровне системы. Эти границы нужно уточнять в каждой команде.

Изучи жизненный цикл изменения: проблема → исследование → решение → постановка → разработка → тестирование → выпуск → проверка результата. Разбери backlog, refinement, планирование, review, retrospective, Definition of Done. Agile - семейство подходов и ценностей, Scrum - конкретный фреймворк. Аналитик не является отдельной обязательной ролью Scrum; должность компании и ответственность внутри фреймворка - разные уровни описания.

Обязательно:

- S06 Бизнес и системный аналитик Что нужно знать · RU. Прочитай обзор целиком. Выпиши, какие действия автора относятся к бизнесу, а какие к системе.
- S01 Scrum Guide · EN. Прочитай Scrum Team, события, артефакты и Definition of Done. Не учи формулировки дословно.

Расширение:

- S77 Kanban · EN. Разбери визуализацию потока, ограничение незавершённой работы и отличие потока от спринтов.
- R02 БА и СА в SA Wiki · RU. Можно использовать как дополнительную русскую памятку о ролях; содержание страницы в этой проверке полностью не извлечено.

Сделай: нарисуй путь задачи «клиент отменяет запись» через команду. Для каждой стадии укажи ответственного, вход, результат и вопрос, который нужно решить. Составь таблицу ролей: кто знает правила отмены, кто согласует срок, кто проверяет контракт, кто подтверждает готовность выпуска.

Самопроверка: кто решает, стоит ли делать функцию? Кто подтверждает бизнес-правило? Что делать, если разработчик нашёл более дешёвое решение? Почему окончание написания ТЗ не означает окончание работы аналитика? Чем «реализовано» отличается от «принято»?

Готово, когда: ты за пять минут объясняешь путь изменения и на каждом этапе показываешь полезный результат своей работы. Не подменяй ответ перечнем должностей и встреч.

J02 Пойми как запрос проходит через систему

Результат: ты читаешь HTTP-запрос и объясняешь, что произошло между интерфейсом и сервером. Ориентир - 14 часов.

Разбери клиент, сервер, frontend, backend, базу данных, домен, DNS, IP, порт, URL и HTTPS. Тебе пока не нужно администрировать сеть. Нужно понимать, где искать причину: запрос вообще не отправился, сервер отказал, данные не найдены или интерфейс неправильно обработал ответ.

В HTTP изучи метод, путь, query-параметры, заголовки, тело и код ответа. Начни с GET, POST, PUT, PATCH, DELETE; коды - 200, 201, 202, 204, 400, 401, 403, 404, 409, 422, 429, 500, 502, 503, 504. Конкретный набор для нашего API согласуем в J12. Не считай любой 4xx виной человека, а любой 5xx доказательством ошибки твоего backend: есть посредники и внешние зависимости.

Обязательно:

- S16 Как работает Интернет · RU. Прочитай объяснение сети, доменных имён и взаимодействия компьютеров.
- S17 Обзор протокола HTTP · RU. Разбери структуру запроса и ответа, клиентскую и серверную стороны.
- S18 Коды состояния HTTP · RU. Прочитай перечисленные выше коды и сравни 401/403, 201/202, 409/422.
- S20 Инструменты разработчика в браузере · EN. Найди Network, Headers, Payload и Response. Научись сохранять пример запроса без секретов.

Сделай: открой Network на публичной странице, отфильтруй Fetch/XHR и выбери один безопасный запрос чтения. Запиши URL, метод, параметры, код и тип ответа. Покажи отдельно, какие данные относятся к HTTP, а какие - к предметной области. Затем открой учебный JSONPlaceholder и сравни запрос конкретного объекта со списком.

Самопроверка: чем URL отличается от IP? Где передать фильтр по дате? Есть ли тело у каждого запроса? Почему 200 ещё не доказывает правильность бизнес-результата? Почему таймаут не доказывает, что операция не выполнилась?

Готово, когда: по примеру запроса ты можешь объяснить, кто к кому обратился, о чём попросил и что получил. Нельзя отправлять запросы на изменение в чужих продуктивных системах ради упражнения.

J03 Выясни задачу заказчика и границы решения

Результат: вместо фразы «нужна запись онлайн» у тебя есть согласованная задача. Ориентир - 16 часов.

Сначала выясняй текущую проблему и желаемый результат. Вопрос «какую кнопку сделать?» обычно преждевременен. Нужны участники процесса, текущий порядок действий, частые исключения, ограничения и способ понять, что стало лучше. Различай факт, мнение, предположение и принятое решение.

Подготовь интервью: цель встречи, участники, открытые вопросы, реальные примеры, спорные места. Проси описать последний конкретный случай. После встречи отправь краткий протокол: решение, открытый вопрос, ответственный и срок ответа. Слова «вроде договорились» не заменяют согласованное правило.

Обязательно:

- S04 От хочу к ТЗ · RU. Прочитай части о сборе, анализе и фиксации требований. Составь собственный список вопросов к учебному центру.
- S05 Работа с требованиями на разных этапах проекта · RU. Посмотри, как требования уточняются по ходу проекта и почему постановка меняется.

Сделай: подготовь 20 вопросов администратору, проведи 30-минутное учебное интервью с ментором или партнёром. Узнай, как считают места, что происходит при отмене и кто исправляет ошибку записи. Напиши Vision на одну-две страницы: проблема, пользователи, желаемый результат, границы первой версии, ограничения, допущения, вопросы. Составь глоссарий минимум из 12 терминов. Договорись, чем «занятие» отличается от «курса», а «клиент» от «учётной записи».

Практический пример: «клиенты часто забывают отменять» - наблюдение, которое стоит подтвердить. «Нужно напоминание» - предложенное решение. «Снизить долю пустых мест» - цель, для которой пока нет измеренного исходного значения. В документе эти три вещи должны стоять отдельно.

Самопроверка: кто принимает спорное правило? Что ты спросишь про последнее свободное место? Как отличить нужную функцию от пожелания одного сотрудника? Что делать, если две роли описывают процесс по-разному? Какие вопросы блокируют проектирование, а какие можно отложить?

Готово, когда: заказчик узнаёт свой процесс в твоём описании и подтверждает границы. Тебе не приходится придумывать ответы за него. Если ментор пока недоступен, продолжай с явно записанными допущениями и вернись к проверке позже.

J04 Научись писать проверяемые требования

Результат: разработчик и тестировщик одинаково понимают, что должна сделать система. Ориентир - 20 часов.

Разбери бизнес-требование, пользовательскую потребность, функциональное требование, бизнес-правило, ограничение и нефункциональное требование. Бизнес-правило «отмена доступна за 12 часов» существует независимо от конкретной кнопки. Функция описывает, как система это правило применяет. НФТ описывает качество и условия работы: скорость, доступность, безопасность, наблюдаемость.

Проверяй требование на однозначность, необходимость, непротиворечивость и возможность проверки. В одном пункте лучше держать одну мысль. Избегай «быстро», «удобно», «при необходимости», «как обычно», если не объяснено, что это означает. Указывай источник, идентификатор, статус и приоритет. Приоритет обсуждается с владельцем продукта; правило MoSCoW полезно только при согласованном смысле категорий.

Обязательно:

- S04 От хочу к ТЗ · RU. Повтори разделы анализа и оформления. Сравни свои требования с вопросами, на которые должно отвечать ТЗ.
- S07 Критерии приемки для пользовательских историй · RU. Разбери связь пользовательского поведения и критериев приёмки.
- S64 Service Level Objectives · EN. Прочитай вводные объяснения индикатора и целевого уровня сервиса; сложные расчёты пока пропусти.

Сделай: напиши не меньше 18 требований проекта: функции, правила, ограничения и три НФТ. Для каждого добавь источник и способ проверки. Исправь формулировки «система работает быстро», «администратор может всё», «запись отменяется при ошибке». Для НФТ скорости укажи действие, границу измерения, процентиль, нагрузку и среду проверки. Учебное число можно предложить, но его нужно пометить как гипотезу для согласования.

Пример: «При попытке отмены менее чем за 12 часов до начала система отказывает в отмене и сохраняет активную запись». Дальше уточни включённость границы, источник времени и поведение администратора. Только после этого правило становится пригодным для реализации.

Самопроверка: где требование, а где решение? Как протестировать «не более 2 секунд» без указания нагрузки? Кто подтверждает приоритет? Что делать, если два требования противоречат друг другу? Может ли одно правило использоваться несколькими функциями?

Готово, когда: QA может предложить проверку каждого требования, а спорные формулировки вынесены в открытые вопросы. Наличие номера FR-001 само по себе не делает текст качественным.

J05 Опиши пользовательские сценарии

Результат: ты умеешь разложить функцию на нормальное поведение, альтернативы и ошибки. Ориентир - 18 часов.

User Story помогает договориться о потребности и ценности. Она не заменяет подробное описание поведения. Use Case удобен, когда нужно последовательно показать взаимодействие участника и системы. Диаграмма Use Case показывает участников и цели, но подробности сценария остаются в тексте.

Изучи акторов, триггер, предусловия, основной поток, альтернативы, исключения и постусловия. Отдельно формулируй критерии приёмки. В формате Given/When/Then опиши исходное состояние, действие и наблюдаемый результат. Не превращай сценарий в инструкцию по внутренним классам backend.

Обязательно:

- S02 INVEST для пользовательских историй · EN. Проверь истории на независимость, обсуждаемость, ценность, оцениваемость, небольшой размер и проверяемость.
- S03 Gherkin Reference · EN. Прочитай Feature, Scenario, Given, When, Then и Scenario Outline. Автоматизация тестов пока не требуется.
- S07 Критерии приемки для пользовательских историй · RU. Возьми один пример и объясни, какие границы поведения он фиксирует.

Расширение:

- S14 UML Use Case в PlantUML · RU. Посмотри акторов и связи Use Case. Не добавляй include/extend, если не можешь объяснить их смысл.

Сделай: напиши 6 историй проекта и разбей слишком крупные. Подготовь 3 Use Case: запись, отмена, просмотр списка администратором. Для записи нужны альтернативы: мест нет, клиент уже записан, занятие началось, произошёл повторный запрос. Добавь минимум 10 критериев приёмки с граничными значениями. Для отмены проверь 12 часов ровно и на секунду меньше.

Пример проверки: дано занятие с одним свободным местом и клиент без активной записи; когда он подтверждает запись; тогда создаётся одна активная запись и остаётся ноль свободных мест. Одновременные запросы двух клиентов пока зафиксируй как отдельный технический риск: подробно решим его в следующем маршруте, а в текущей постановке укажем обязательный инвариант вместимости.

Самопроверка: что важнее в истории - роль или результат? Чем альтернативный поток отличается от нового Use Case? Что система сохраняет после отказа? Как обнаружить пропущенный сценарий? Какие критерии относятся именно к этой истории?

Контрольная точка 1: защити задачу перед «заказчиком» за 20 минут. Покажи Vision, требования и три сценария. Ментор меняет одно правило; ты называешь документы, которые нужно обновить. Если объяснить связь не получается, вернись к J03-J05.

J06 Нарисуй бизнес процесс в BPMN

Результат: схема объясняет, кто что делает и чем заканчиваются разные варианты процесса. Ориентир - 18 часов.

BPMN нужна, чтобы обсуждать процесс с участниками и находить пробелы. Начни с пула, дорожек, задачи, стартового и конечного событий, sequence flow, message flow, исключающего и параллельного шлюзов. Потом добавь сообщение, таймер и граничное событие. Каждую новую фигуру используй только после того, как можешь объяснить её поведение.

Пул обычно обозначает отдельного участника взаимодействия; дорожки распределяют ответственность внутри него. Поток последовательности не пересекает границы пулов, обмен между пулами показывают сообщениями. XOR выбирает одну ветку, AND запускает параллельные; их слияния тоже ведут себя по-разному. Хорошая схема не должна оставлять непонятных зависаний.

Обязательно:

- S09 BPMN Reference · EN. Изучи задачи, события, потоки, XOR и AND. Пройди приведённые примеры по шагам как участник процесса.
- S08 Как сделать BPMN диаграмму чуточку лучше · RU. Разбери ошибки из практики и проверь по ним свою первую схему.

Видео и расширение:

- V01 Рисуем схему с нуля BPMN Практика в diagrams net · RU. Попробуй повторить построение схемы, затем закрой видео и нарисуй свой процесс. Видео найдено в индексе, воспроизведение отдельно не проверено.
- S10 BPMN Examples · EN. Выбери примеры с сообщением и таймером. Разбери, что происходит, если событие не наступает.
- S11 BPMN 2.0.2 · EN. Справочник семантики для спорных случаев; весь стандарт на этом этапе читать не нужно.
- R03 BPMN в SA Wiki · RU. Дополнительная русская памятка. Не используй её как единственный источник правил нотации.

Сделай: нарисуй AS IS записи через администратора и TO BE с личным кабинетом. Отметь начало, результат и ответственность на каждом шаге. Вынеси внешний сервис уведомлений в отдельный пул. Покажи отказ из-за отсутствия мест и отдельный результат ошибки доставки. Пройди по схеме три сценария, включая неуспешный. Опиши двумя абзацами, какая проблема исчезает и какие ручные действия остаются.

Самопроверка: может ли поток последовательности выйти в другой пул? Что произойдёт, если объединить параллельные ветки XOR-шлюзом? Где схема описывает работу человека, а где системы? Какой результат у каждого конца процесса? Нужно ли рисовать HTTP-вызовы в бизнес-процессе?

Готово, когда: другой человек проводит сценарий по схеме и получает тот же результат, что в Use Case. BPMN не должна противоречить требованиям и превращаться в картинку всех технических деталей.

J07 Покажи взаимодействия и состояния в UML

Результат: ты умеешь показать порядок вызовов и жизненный цикл объекта. Ориентир - 18 часов.

Диаграмма последовательности отвечает, кто с кем взаимодействует и в каком порядке. Изучи lifeline, сообщение, ответ, alt, opt, loop и примечания. Отделяй вызов от ответа и подписывай условия альтернатив. Стрелка сама по себе не объясняет, что делать после таймаута: нужен сценарий.

Диаграмма состояний отвечает, в каких состояниях бывает объект и какие события разрешают переход. Начни с небольшой модели: запись активна или отменена. Не добавляй «успешно», «ошибка», «обработано» без определения. Транспортная ошибка вызова и бизнес-статус объекта не одно и то же.

Обязательно:

- S12 UML Sequence в PlantUML · RU. Разбери участников, вызовы, ответы и группы alt/opt/loop. Сначала прочитай пример, затем воспроизведи его самостоятельно.
- S13 UML State в PlantUML · RU. Изучи состояния, переходы, начальное и конечное псевдосостояния.

Расширение:

- V02 Activity Diagram построение и применение · RU. Посмотри 02:29-28:36: элементы Activity Diagram, построение, ошибки и сравнение с BPMN. После просмотра объясни, нужен ли этот вид диаграммы в твоём проекте.
- S15 UML 2.5.1 · EN. Используй как нормативный справочник UML. Синтаксис PlantUML и семантика UML - разные уровни.

Сделай: нарисуй sequence для записи и отмены: клиент, frontend, backend, база, сервис уведомлений. Укажи проверку прав и бизнес-правил, запись результата и обработку ошибки уведомления. Подготовь таблицу переходов с колонками «исходное состояние», «событие», «условие», «новое состояние», «побочные эффекты». Добавь запрещённые переходы и поведение повторной отмены.

Самопроверка: что видит клиент до ответа внешней системы? Из какого состояния разрешена отмена? Можно ли восстановить отменённую запись или нужна новая? Почему статус уведомления не должен менять статус записи автоматически? Что проверит QA по таблице переходов?

Готово, когда: таблица переходов, sequence и текст сценария согласованы. Если в одном месте появился новый статус, ты обновил его описание в остальных работах. Исходники PlantUML хранятся рядом с экспортом диаграмм.

J08 Спроектируй данные

Результат: у тебя есть понятная модель сущностей, связей и ограничений. Ориентир - 22 часа.

Начни с предметной области, а потом выбирай таблицы. Концептуальная модель объясняет сущности и связи; логическая - атрибуты, ключи и кардинальности; физическая - реализацию в конкретной СУБД. Не превращай каждый экран в отдельную таблицу и не копируй JSON-ответ в структуру базы без анализа.

Изучи первичный и внешний ключ, естественный и суррогатный ключ, связи 1:1, 1:N и M:N, обязательность связи, NULL, UNIQUE, CHECK, NOT NULL. Разберись с нормализацией до третьей нормальной формы на небольшом примере: одно значение, зависимость от ключа, отсутствие лишней зависимости через неключевой атрибут. Нормализация помогает уменьшить противоречия, но конкретные решения зависят от доступа к данным.

Для времени различай момент события и локальное время расписания. Для денег в будущих задачах используй точный десятичный тип или целые минимальные единицы с валютой; двоичные числа с плавающей точкой не являются безопасной универсальной моделью денежных сумм.

Обязательно:

- V05 CS50 SQL Week 2 Designing · EN. Лекция Designing: сущности, связи, ключи, ограничения и проектирование схемы. Если удобнее текст, используй Notes на той же странице. Примеры курса могут использовать SQLite.
- S23 Ограничения данных в PostgreSQL · RU. Прочитай NOT NULL, CHECK, UNIQUE, PRIMARY KEY и FOREIGN KEY. Сопоставь каждый механизм с правилом проекта.

Расширение:

- V04 CS50 SQL Week 1 Relating · EN. Лекция Relating помогает увидеть связи и необходимость JOIN.
- S24 PostgreSQL Tutorial · EN. Используй главы учебника о создании таблиц, внешних ключах и транзакциях.

Сделай: подготовь ERD для clients, sessions, bookings, notification_attempts. Дай каждому атрибуту описание, тип, обязательность, пример и источник. Покажи, что один клиент имеет много записей, а одно занятие - много записей. Объясни, где хранится вместимость, кто является источником истины о свободных местах и что произойдёт при удалении занятия.

Опиши правило одной активной записи на пару клиент-занятие. Обычный UNIQUE(client_id, session_id) также запрещает повторную запись после отмены. Выбери явно: переиспользование той же строки с историей переходов либо новая строка и уникальность только активных записей. Для первого проекта допустимо выбрать первый вариант и зафиксировать его ограничения.

Самопроверка: может ли внешняя связь указывать на несуществующий объект? Чем NULL отличается от пустой строки? Что потеряется при каскадном удалении? Зачем словарь данных, если есть картинка? Как правило вместимости связано с конкурентными запросами?

Готово, когда: по модели можно создать таблицы, а для каждого бизнес-правила указано, где оно обеспечивается. Правило «активных записей не больше вместимости» требует согласованного алгоритма, а не только красивой ERD.

J09 Освой SQL на одной таблице

Результат: ты самостоятельно получаешь нужную выборку и проверяешь её смысл. Ориентир - 24 часа.

Нужны SELECT, FROM, WHERE, ORDER BY, LIMIT, DISTINCT, AND/OR/NOT, IN, BETWEEN, LIKE, IS NULL, CASE, агрегатные функции, GROUP BY и HAVING. Дополнительно разбери INSERT, UPDATE, DELETE и простую транзакцию BEGIN/COMMIT/ROLLBACK. Практикуй изменение только на своей учебной базе.

Сразу учись читать вопрос к данным: что является одной строкой результата, какие объекты включаются, за какой период и что делать с отсутствующими значениями. Если эти условия не выяснены, синтаксически верный запрос всё равно может отвечать неверно.

Обязательно:

- S22 Язык SQL в документации PostgreSQL · RU. Пройди создание, наполнение, выборку, агрегаты, изменение и удаление строк. Повторяй команды в своей базе.
- S28 SQLBolt · EN. Пройди уроки 1-4, повторение SELECT и уроки об агрегатах. Синтаксические отличия сверяй с PostgreSQL.

Альтернатива и видео:

- S29 Интерактивный тренажер по SQL · RU. Русский тренажёр: блоки с одной таблицей и агрегатами. В учебных заданиях возможен другой SQL-диалект. Поиском курс подтверждён, прямое открытие в проверке не сработало.
- V03 CS50 SQL Week 0 Querying · EN. Querying: SELECT, фильтрация, сортировка и агрегаты. Можно читать Notes вместо просмотра всей лекции.

Сделай: заполни таблицу занятий минимум 20 строками: разные даты, вместимость, отменённые занятия, нулевое количество записей. Напиши 15 запросов: будущие занятия; занятия за период; поиск по части названия; пять ближайших; число занятий по дням; средняя вместимость; дни с более чем тремя занятиями; строки с отсутствующим описанием. Перед каждым запросом напиши ожидаемые строки или число строк, затем сравни результат.

Отдельно покажи разницу COUNT(*) и COUNT(description), WHERE и HAVING, AND/OR со скобками. Выполни UPDATE внутри транзакции и откати. Объясни, почему отсутствие WHERE при изменении опасно.

Самопроверка: почему сравнение `= NULL` не подходит? Как определить порядок строк? Включает ли BETWEEN границы? Почему фильтр по последнему дню может потерять часть записей с временем? Чем запрос количества отличается от списка объектов?

Готово, когда: решаешь 8 из 10 новых небольших задач без готовых ответов, проверяешь результат на тестовых данных и объясняешь каждую часть запроса. При ошибке сначала проверь данные и условия, затем синтаксис.

J10 Соедини таблицы и проверь качество данных

Результат: ты строишь выборки по связанным таблицам и замечаешь неверные результаты. Ориентир - 24 часа.

Изучи INNER JOIN, LEFT JOIN, связь условий ON и WHERE, UNION/UNION ALL, подзапросы, EXISTS/NOT EXISTS и простые CTE. Понимай кардинальность результата: JOIN с несколькими подходящими строками размножает строки. DISTINCT не должен маскировать неправильное соединение.

Научись отвечать не только на вопрос «сколько записей», но и «какие данные нарушают правило». Это повседневная работа аналитика: проверить гипотезу, найти затронутые объекты, объяснить результат QA. Оконные функции можно оставить на второй маршрут, если базовые соединения ещё неустойчивы.

Обязательно:

- S22 Язык SQL в документации PostgreSQL · RU. Прочитай соединения и агрегатные функции, повтори на таблицах своего проекта.
- S28 SQLBolt · EN. Пройди multi-table queries, outer joins и NULL.
- V04 CS50 SQL Week 1 Relating · EN. Разбери связи, соединения и подзапросы. Перепиши один пример под свой проект.

Сделай: добавь клиентов, записи и попытки уведомлений. Специально внеси занятие без записей, клиента без бронирований, два уведомления одной записи и отменённую запись. Напиши 12 запросов: список записей с именами; занятия без записей; клиенты без активных записей; заполненность каждого занятия; последние ошибки уведомлений; нарушения уникальности; число клиентов за период; отмены по дням. Для заполненности учитывай только активные записи и сохраняй занятия с нулём.

Отдельная ловушка: соедини bookings с notification_attempts и посчитай записи. Покажи завышение из-за нескольких попыток, затем исправь запрос через предварительную агрегацию или подходящее условие. Объясни, какое множество объектов считаешь.

Самопроверка: почему LEFT JOIN с условием по правой таблице в WHERE может убрать строки без пары? Когда EXISTS понятнее JOIN? Чем UNION отличается от UNION ALL? Как проверить, что отчёт не удвоил клиентов? Нужно ли сохранять строки без данных?

Контрольная точка 2: 45 минут на новую схему из трёх таблиц: объяснить связи, написать 3 выборки и найти ошибку в чужом JOIN. Плюс 15 минут на защиту ERD проекта. Минимум для перехода - корректные выборки и объяснение кардинальности, а не скорость набора SQL.

J11 Разберись в форматах сообщений

Результат: ты читаешь JSON и XML, описываешь ограничения данных и не путаешь пример со схемой. Ориентир - 14 часов.

В JSON изучи объект, массив, строку, число, boolean и null. Отличай отсутствующее поле от поля со значением null, а пустой массив - от отсутствия коллекции. Одинаковое на вид значение может иметь разный тип: `"100"` не равно числу `100` в контракте.

Для каждого поля фиксируй тип, обязательность, допустимые значения, формат и смысл. JSON Schema описывает проверяемую структуру сообщения. Пример показывает один допустимый экземпляр; по нему нельзя понять все ограничения. Для XML нужны элементы, атрибуты, namespace и базовое представление о XSD. Подробную разработку SOAP-сервисов пока оставь на специализацию.

Обязательно:

- S19 JSON · RU. Изучи типы и ограничения формата JSON. Обрати внимание на кавычки, отсутствие комментариев и завершающих запятых.
- S37 Creating your first JSON Schema · EN. Пройди первую схему: type, properties, required, ограничения и вложенные структуры.

Справочник:

- S41 XML 1.0 · EN. Посмотри синтаксис XML, элементы и атрибуты. Не читай стандарт целиком.

Сделай: опиши JSON занятия и записи. Для даты начала используй однозначное представление времени с зоной или смещением и запиши правило интерпретации. Подготовь один корректный и шесть некорректных примеров: неверный тип, неизвестный статус, отсутствующий обязательный идентификатор, отрицательная вместимость, пустое имя, неверная дата. В JSON Schema зафиксируй минимум type, required, enum и ограничения чисел/строк.

Затем представь маленький объект уведомления в XML и объясни, какие сведения передал атрибутами, а какие элементами. Не пытайся доказать, что один формат всегда лучше другого.

Самопроверка: является ли JSON протоколом? Можно ли по одному примеру определить обязательность? Чем JSON Schema отличается от OpenAPI? Почему null не стоит трактовать как «не изменять» без явного соглашения? Какое поведение у неизвестного поля?

Готово, когда: независимый человек по твоему описанию отличает корректное сообщение от некорректного. Настройка валидатора тоже важна: например, проверка `format` не во всех реализациях включена как обязательное ограничение.

J12 Спроектируй API первой версии

Результат: по контракту можно реализовать клиент и сервер, не угадывая основные правила. Ориентир - 24 часа.

REST - архитектурный стиль, HTTP - протокол. Не называй любой JSON поверх HTTP строгим REST. Для практики нам достаточно последовательного ресурсного HTTP API: занятия, записи, их представления и операции. Изучи различие path/query/header/body, безопасных и идемпотентных методов, создания и изменения ресурса, фильтрации и пагинации.

Начни с GET /sessions, GET /sessions/{sessionId}, POST /bookings, GET /bookings/{bookingId}. Для отмены выбери и обоснуй один вариант: отдельное действие или изменение состояния. Не меняй его между документами. Клиентский идентификатор владельца не заменяет проверку прав: принадлежность определяется через доверенный контекст пользователя.

Обязательно:

- S17 Обзор протокола HTTP · RU. Повтори методы, заголовки и структуру сообщений.
- S18 Коды состояния HTTP · RU. Выбери коды для создания, отсутствия объекта, отказа в доступе и конфликта бизнес-состояния.
- S38 Проектирование спецификации OpenAPI · RU. Прочитай подход к проектированию контракта до реализации. На этом шаге сосредоточься на структуре и согласованности API.

Расширение:

- S73 RFC 9110 HTTP Semantics · EN. Точечно прочитай разделы о safe/idempotent и выбранных методах. Идемпотентность относится к ожидаемому эффекту, а не к одинаковому телу ответа.
- S72 RFC 9457 Problem Details for HTTP APIs · EN. Познакомься со стандартным представлением ошибок. Для проекта можно выбрать упрощённый согласованный формат.

Сделай: опиши 5 методов с целью, правами, запросом, успешным ответом, ошибками и примерами. У каждого поля укажи смысл, тип и обязательность отдельно для запроса и ответа. Для списка добавь порядок сортировки, размер страницы, ограничения фильтров и поведение пустой выборки. Для создания - проверку вместимости, дубликата и уже начавшегося занятия.

Составь таблицу ошибок: условие, HTTP-код, бизнес-код, сообщение и действие клиента. Разведи «мест нет», «неверная дата», «нет доступа», «сервис временно недоступен». Не привязывай фронт к тексту сообщения, если для обработки нужен стабильный код.

Самопроверка: почему GET не должен создавать запись? Всегда ли повторный POST безопасен? Что означает 202? Чем 404 отличается от пустого списка? Что произойдёт, если клиент отправит неизвестный статус? Зачем контракту сортировка при пагинации?

Готово, когда: разработчик может написать обработчик, а QA - позитивные, негативные и граничные проверки. Поведение при конкуренции описано как обязательное правило и передано на техническое согласование, даже если Junior ещё не выбирает механизм блокировок самостоятельно.

J13 Оформи OpenAPI и проверь запросы в Postman

Результат: контракт существует в машиночитаемом виде, а примеры запросов можно повторить. Ориентир - 24 часа.

OpenAPI описывает HTTP API: пути, операции, параметры, схемы, ответы и способы авторизации. Swagger - семейство инструментов вокруг спецификации. В этом маршруте фиксируем OpenAPI 3.1.1 как учебную версию; это не заявление о самой новой версии стандарта. Не смешивай её синтаксис с примерами для 3.0 без проверки.

Разбери info, servers, paths, components/schemas, operationId, parameters, requestBody, responses и security. В OpenAPI 3.1 null описывается средствами JSON Schema, например `type: [string, "null"]`; знакомое по 3.0 `nullable: true` не следует переносить механически. Поле required в объекте - список имён обязательных свойств, а required у параметра - отдельный boolean.

Обязательно:

- S38 Проектирование спецификации OpenAPI · RU. Прочитай статью и воспроизведи подход на своих методах. Не копируй решения без привязки к кейсу.
- S34 OpenAPI 3.1.1 · EN. Используй разделы OpenAPI Object, Operation, Parameter, Request Body, Responses и Schema Object.
- S32 Postman Quick Start · EN. Пройди первый запрос и изучи коллекции и переменные окружения.
- S36 Swagger Editor · EN. Открой YAML, проверь синтаксис и представление контракта. Проверка синтаксиса не заменяет проверку бизнес-логики.

Сделай: перенеси методы J12 в один openapi.yaml. Вынеси общие схемы, сделай примеры запросов и ответов, опиши ошибки и безопасность. Импортируй контракт в Postman. Создай переменную baseUrl и запросы на чтение, создание и отмену. Учебный публичный API используй только для освоения инструмента: JSONPlaceholder имитирует изменение данных, а не реализует правила нашего сервиса.

Если собственного backend пока нет, разделяй два результата: коллекция демонстрационных запросов к учебному API и проектный контракт с подготовленными примерами/моками. Не пиши, что бизнес-проверки сервиса прошли на реальном сервере, если сервер отсутствует.

Добавь проверки HTTP-кода, обязательного идентификатора и типа данных. Проверь один ошибочный запрос. Настройки аккаунта и облачного режима Postman могут меняться; для передачи работ достаточно экспортируемых файлов без токенов.

Самопроверка: чем схема отличается от example? Почему удачный импорт ещё не доказывает полноту контракта? Где документировать 409? Что меняется при nullable-поле в 3.1? Чем контрактный мок отличается от работающего backend?

Готово, когда: YAML разбирается выбранным инструментом, все методы из постановки в нём присутствуют, примеры согласованы со схемами, секретов в экспорте нет.

J14 Опиши интеграцию с внешней системой

Результат: ты понимаешь обмен между системами и описываешь не только успешный ответ. Ориентир - 18 часов.

Синхронный вызов предполагает ожидание ответа в рамках взаимодействия. Асинхронная обработка отделяет принятие работы от её завершения. Это не означает, что HTTP всегда синхронный на уровне бизнес-процесса, а брокер автоматически делает решение надёжным.

Изучи API, файлы, обмен через общую БД и сообщения как разные варианты интеграции. Для первой работы подробно разбери webhook: внешняя система сама вызывает наш endpoint с событием. Нужны идентификатор события, время, тип, ссылка на объект, правила доступа, подтверждение приёма и обработка повторов. Ответ «получили» и результат бизнес-обработки могут быть разными фактами.

Обязательно:

- S53 Webhooks · EN. Прочитай устройство endpoint, обработку дубликатов, порядок событий и подпись. Это пример Stripe, а не универсальные правила каждого партнёра.
- V07 Kafka 101 Events · EN. Посмотри вводный урок об events; прочитай сопровождающий текст. Нужны события, producer и consumer без настройки кластера.

Расширение:

- S54 RabbitMQ Tutorials · EN. Посмотри примеры очереди и publish/subscribe. Пока объясни смысл, запускать все языковые реализации не нужно.
- S42 SOAP 1.2 Messaging Framework · EN. Узнай, из чего состоит SOAP Envelope и Fault. WSDL и XSD понадобятся, если выберешь enterprise-интеграции.

Сделай: опиши обмен с уведомлениями: запрос на отправку, ответ о принятии и callback о доставке. Подготовь таблицу маппинга с источником, целевым полем, преобразованием, обязательностью и примером. Учти неизвестную запись, повтор события, неверную подпись и неизвестный статус. На sequence отдели сохранение записи клиента от отправки уведомления.

Сформулируй решение для сбоя: запись остаётся активной, состояние уведомления показывает ошибку, повтор отправки контролируется отдельно. Не обещай, что уведомление будет отправлено «когда-нибудь»: укажи, кто замечает исчерпание попыток и что делает.

Самопроверка: чем webhook отличается от polling? Кто инициирует callback? Можно ли повторить запрос после таймаута? Как распознать повтор одного события? Что делать с событием о несуществующем объекте? Почему ошибка SMS не равна ошибке бронирования?

Контрольная точка 3: защити API и интеграцию перед «тимлидом» за 30-40 минут. По запросу покажи поле в контракте, его источник в данных и проверку QA. Отдельно разберите потерянный ответ и повторный callback. Глубокие механизмы надёжности относятся к следующему маршруту, но риск нельзя оставить неописанным.

J15 Опиши интерфейс и доступ к данным

Результат: поведение экрана, API и разрешений не противоречит друг другу. Ориентир - 16 часов.

Аналитику нужен прототип, чтобы согласовать действия и состояния. Художественная отрисовка необязательна. Для каждого экрана опиши загрузку, данные, пустое состояние, ошибку, валидацию, повтор действия и недоступность функции. Укажи, какие значения вводит человек, какие приходят с backend и какие рассчитываются.

Различай аутентификацию «кто ты» и авторизацию «что тебе разрешено». Роль и владение конкретным объектом проверяются на сервере. Если пользователь подставил чужой bookingId, скрытая кнопка в интерфейсе не защищает данные. В логах и примерах не нужны действующие токены и реальные персональные данные.

Обязательно:

- S76 Forms Tutorial · EN. Прочитай подписи полей, инструкции, валидацию и уведомления об ошибках. Перенеси принципы на свой прототип.
- S62 Authorization Cheat Sheet · EN. Изучи least privilege, deny by default и проверку прав на каждом запросе.
- S61 OWASP API Security Top 10 2023 · EN. Прочитай риск Broken Object Level Authorization и объясни его на чужом bookingId.

Сделай: нарисуй три экрана: расписание, мои записи, список участников занятия для администратора. Для каждого перечисли состояния и связь с API. Составь матрицу прав клиент/администратор/внешний сервис. Добавь проверки чтения чужой записи и попытки выполнить административную операцию обычным пользователем.

Отдельно опиши, что увидит клиент после двойного нажатия «Записаться» и потери сети. Временная блокировка кнопки уменьшает число запросов, но сервер всё равно должен обеспечивать согласованные правила повторной операции. Начни с точного описания ожидаемого поведения, затем согласуй механизм с разработчиком.

Самопроверка: какие проверки нужны на backend даже при валидации frontend? Можно ли доверять clientId из тела запроса? Что показать при пустом списке? Чем ошибка поля отличается от недоступности сервиса? Какие данные не стоит включать в скриншот дефекта?

Готово, когда: по прототипу понятны действия, по матрице - права, а по таблице состояний - поведение при ошибках. Проверка доступа присутствует в сценариях API, а не только в описании интерфейса.

J16 Собери постановку и научись менять требования

Результат: другой участник команды может взять твою постановку и начать работу. Ориентир - 18 часов.

Требования не обязаны жить в одном огромном файле. Важнее единая точка входа, согласованность и известный статус. В постановке должно быть понятно: зачем изменение, какие границы, как было и как станет, какие правила действуют, что меняется в данных, API и интерфейсе, какие ошибки ожидаются и как принять результат.

Изучи декомпозицию на небольшие проверяемые изменения, связь требования с реализацией и проверкой, журнал решений и управление версиями. Решение из чата нужно перенести в действующую спецификацию. Иначе команда реализует разные версии одной функции.

Обязательно:

- S05 Работа с требованиями на разных этапах проекта · RU. Прочитай работу с требованиями на разных стадиях. Обрати внимание на сопровождение и согласование изменений.
- S39 Как мы ломали и собирали API заново · RU. Разбери опыт изменения API: кто должен знать о контракте и почему последствия выходят за рамки одного метода.
- S67 Pro Git · RU. Повтори просмотр изменений и историю коммитов; сделай отдельную версию постановки после ревью.

Сделай: собери постановку «Самостоятельная отмена записи». Используй шаблон в приложении. Добавь ссылки на требования, сценарий, таблицу переходов, данные, API, прототип и приёмку. Составь маппинг полей фронт → API → данные → ответ, включая вычисляемые значения и источник пользователя.

Затем получи изменение от ментора: теперь администратор может отменить запись после пользовательского дедлайна, но обязан указать причину. Подготовь impact analysis: что затронуто, что меняется, кто согласует, какие проверки добавляются. Обнови связанную документацию, а не только один абзац.

Самопроверка: что в документе является актуальной версией? Как узнать, какие тесты затронуты новым правилом? Нужно ли согласование frontend? Где хранится причина отмены? Кто подтверждает спорное решение? Что можно вынести в общую документацию, чтобы не копировать?

Готово, когда: ментор в роли разработчика находит входные данные, алгоритм, ограничения и критерии приёмки. Открытые вопросы видны отдельно и не спрятаны в тексте. Нет противоречий между API, моделью и интерфейсом.

J17 Проверь результат и помоги команде разобраться с ошибкой

Результат: ты умеешь принять свою фичу и описать проблему так, чтобы её можно было воспроизвести. Ориентир - 16 часов.

Разбери позитивные, негативные, граничные и интеграционные проверки. Уровни тестирования и виды проверок - разные классификации. Аналитик не заменяет QA, но должен понимать, как подтвердить бизнес-результат, и замечать неоднозначности до разработки.

Освой формат дефекта: среда, версия, предусловия, шаги, ожидаемый результат со ссылкой на требование, фактический результат и доказательства. Различай дефект реализации, ошибку тестовых данных и недоопределённое требование. В последнем случае сначала нужно согласовать поведение.

Обязательно:

- S03 Gherkin Reference · EN. Используй Given/When/Then для нескольких приёмочных сценариев.
- S32 Postman Quick Start · EN. Повтори запросы и изучи проверки ответа, доступные в документации Postman.
- S63 Logging Cheat Sheet · EN. Прочитай, какие события полезно логировать и какие данные нельзя бесконтрольно записывать.

Сделай: составь 20 проверок отмены и записи, включая права, границы времени, дубликат и сбой уведомления. Для каждой укажи требование и ожидаемое состояние данных. Оформи 3 дефекта на искусственно подготовленных примерах. Разбери цепочку логов по одному requestId: запрос принят, запись сохранена, отправка уведомления завершилась ошибкой. Объясни, какой бизнес-результат уже состоялся и что ещё требует действия.

Если backend отсутствует, честно оформи план проверок и разбор примеров, а не протокол успешной приёмки работающего продукта. Для учебной интеграции можно использовать подготовленные запросы и ответы как тестовые фикстуры.

Самопроверка: какую границу времени проверять? Как отличить повторное уведомление от повторного создания записи? Что означает отсутствие строки в логах? Когда нужен correlationId? Какое утверждение нельзя сделать по одному HTTP-коду? Когда нужно исправлять ТЗ, а когда реализацию?

Готово, когда: по дефекту можно повторить проблему, ожидаемый результат следует из требований, а вывод не шире доступных доказательств.

J18 Защити проект и подготовь портфолио

Результат: ты показываешь связанную аналитическую работу и решаешь новую небольшую задачу. Ориентир - 28 часов.

Сначала проведи собственное ревью. Открой одно требование и пройди его через сценарий, BPMN, статусы, данные, API и проверку. Если на переходе приходится что-то додумывать, поправь комплект. Портфолио должно показывать ход мысли и качество работы, а не количество диаграмм.

Обязательно:

- S74 Как мы унифицировали техническое интервью аналитика · RU. Прочитай, как одна команда строит техническое интервью. Используй как пример формата; это не единый стандарт всех работодателей.
- S04 От хочу к ТЗ · RU. Сверь полный путь от потребности до сопровождения с собственным проектом.

Сдай комплект: README с контекстом и навигацией; Vision и глоссарий; каталог требований; три Use Case и критерии; AS IS/TO BE; две sequence; состояния и переходы; ERD и словарь данных; SQL с тестовыми данными; OpenAPI; Postman; интеграция и маппинг; прототипы и права; постановка фичи; приёмка и журнал решений.

Защита состоит из трёх частей. За 20 минут объясни заказчику проблему и результат. За 40 минут обсуди с тимлидом данные, методы, ошибки и ограничения. За 30 минут вместе с QA проверь границы и противоречия. Ментор выбирает одну новую задачу: запретить запись на отменённое занятие, добавить фильтр по тренеру или изменить срок отмены. За 60-90 минут подготовь уточняющие вопросы, краткую постановку и список затронутых работ.

На техническом моке реши 3 SQL-задачи, прочитай незнакомый JSON, предложи API и покажи sequence. На HR-моке расскажи о себе за 2-3 минуты: прошлый опыт, почему аналитика, что сделал в проекте, где потребовалась помощь и какую работу ищешь. Учебный проект называй учебным; прежний опыт можно связать с навыками, но не заменять вымышленным коммерческим стажем.

Критерий окончания: по общей шкале из начала гида нет оценки ниже 2; все критичные замечания исправлены; новую задачу ты решаешь с уточняющими вопросами и локальным ревью. Знание определений без возможности применить их не засчитывается.

Для поиска работы сделай резюме, короткое портфолио и таблицу откликов: вакансия, соответствие обязательным требованиям, дата, этап, обратная связь, следующий шаг. Раз в две недели смотри, где теряются переходы. Малое число приглашений и провал SQL на техэтапе требуют разных изменений. Зарплата и срок оффера не являются учебным критерием готовности.

Приложение А Шаблон постановки одной фичи

Копируй структуру и заполняй только нужные разделы. Если раздел не применяется, коротко объясни почему. Не оставляй пустые заголовки ради объёма.

1. **Контекст и цель.** Кто столкнулся с проблемой, как работает сейчас, какой результат нужен, чем его проверим.
2. **Границы.** Что делаем в этой версии, что отложено, зависимости и ограничения.
3. **Термины и источники.** Глоссарий, ссылка на встречу/решение, согласующие лица.
4. **Бизнес-правила.** Условия, формулы, границы, исключения и источник времени.
5. **Сценарии.** Участники, предусловия, триггер, основной поток, альтернативы, постусловия.
6. **Системное поведение.** Алгоритм, sequence, статусы, переходы, повторы и ошибки.
7. **Данные.** Сущности, поля, ограничения, история и удаление.
8. **Контракты.** Методы, поля, примеры, ошибки, авторизация и маппинг.
9. **Интерфейс.** Состояния экранов, валидация, недоступность и сообщения.
10. **НФТ.** Скорость при заданной нагрузке, права, аудит и диагностические сведения.
11. **Приёмка.** Проверяемые сценарии, граничные условия и тестовые данные.
12. **Изменения и вопросы.** Версия, решения, нерешённые вопросы, согласования.

Небольшой пример отмены

Правило: клиент может отменить собственную активную запись, если до начала занятия осталось не менее 12 часов. Время сравнивает сервер. Правило границы включительное. После успешной отмены запись получает состояние CANCELLED; активная вместимость освобождается. История операции сохраняется.

Повтор: повторная отмена уже отменённой собственной записи возвращает согласованное представление CANCELLED без второго освобождения места. Конкретный HTTP-код фиксируется в контракте. **Отказ:** чужая запись недоступна; поздняя отмена возвращает бизнес-ошибку и не меняет состояние. Административное исключение оформляется отдельным правилом после J16.

Проверки: 12 часов ровно; на секунду меньше; уже отменено; чужой объект; занятие уже началось; уведомление не доставлено. Для последнего случая успешная отмена сохраняется. Детали конкурентного изменения вместимости согласуются с разработчиком, а требуемый результат неизменен: место освобождается ровно один раз.

Приложение Б Вопросы с ориентирами для ответа

Сначала отвечай вслух без текста. Затем сравни смысл. Дословное совпадение не требуется.

Вопрос	Что должно быть в хорошем ответе
С чего начать новую функцию	Потребность, участники, текущий процесс, ограничения, критерий результата и вопросы
User Story достаточно для разработки	Не всегда; нужны детали поведения, правила и критерии приёмки
Зачем отдельно бизнес-правило	Оно может использоваться несколькими сценариями и не зависеть от интерфейса
Что даёт AS IS	Общую картину текущей работы, проблем и исключений, а не только будущий экран
Sequence или BPMN	Sequence - взаимодействия во времени; BPMN - процесс и ответственность участников
Почему нужны запрещённые переходы	Они фиксируют поведение при неверной последовательности действий
Чем внешний ключ отличается от обычного поля	Обеспечивает ссылочную целостность по заданному правилу
Почему NULL не равен пустой строке	Это различающиеся значения/отсутствие значения; сравнения и агрегаты ведут себя иначе
Откуда лишние строки после JOIN	Несколько соответствий и выбранная кардинальность связей
WHERE или HAVING	Фильтр строк до группировки или групп после агрегации
Что гарантирует пример JSON	Показывает один экземпляр; не задаёт полностью контракт и обязательность
Что означает идемпотентность	Повтор не должен дополнительно менять ожидаемый эффект; ответ может отличаться
Можно ли после таймаута считать создание неуспешным	Нет; результат мог сохраниться до потери ответа
Почему скрытая кнопка не защита	Запрос можно отправить напрямую; права проверяет сервер
Зачем матрица трассировки	Проследить потребность до сценария, контракта и проверки, найти последствия изменения
Что делать с решением из чата	Зафиксировать принятое решение и обновить актуальную спецификацию
Что доказывает контрактный мок	Согласованное поведение примера/контракта, но не реальную реализацию и её устойчивость
Почему сбой SMS не отменяет запись	Результат бронирования и доставка уведомления - разные состояния и ответственности

Приложение В Как использовать ИИ в обучении

ИИ полезен, чтобы объяснить непонятный фрагмент, придумать контрпример и сыграть роль заказчика. Перед просьбой написать решение попробуй сделать его сам. Проверяй синтаксис инструментом, определения - документацией, бизнес-правила - у владельца процесса. Уверенный ответ модели не является источником требования.

Рабочий запрос для самопроверки: «Ниже моя постановка учебной отмены записи. Проверь противоречия между правилом 12 часов, статусами и API. Не переписывай документ. Дай список конкретных вопросов, укажи фрагмент и объясни риск. Не добавляй бизнес-правил от себя».

Для устной тренировки: «Ты заказчик учебного центра. Задавай по одному вопросу о моей схеме записи. Не подсказывай ответ до моей попытки. После пяти вопросов дай обратную связь по полноте и ясности».

Не передавай в сторонний сервис секреты и данные работодателя. На проверке обязательно повтори ключевую часть самостоятельно: напиши запрос, объясни поле и внеси изменение. В противном случае результат отражает качество подсказки, а не твой навык.

Приложение Г Куда идти после Junior

Если проект защищён, переходи ко второму гиду. Там будет тот же сервис, но уже с неопределённостью, конкуренцией и ограничениями работающей системы. Перед этим ещё раз проверь SQL, требования, статусы и API. Kubernetes, глубокая настройка Kafka, сложные алгоритмы и реализация frontend не нужны как обязательный барьер к первому аналитическому проекту.

Основная база этого маршрута доступна без покупки профессионального курса. Некоторые инструменты требуют установку или аккаунт, а условия сервисов могут меняться. Если дополнительный курс недоступен, используй указанные документацию и альтернативу. Книги Вигерса о требованиях и Коберна о Use Case можно добавить позже как углубление через официального издателя; обязательной покупки для прохождения здесь нет.

Реестр источников этого маршрута

Ссылки проверялись 16 октября 2026 года. «Открыта» означает, что удалось получить страницу и её содержимое; это не обещание вечной доступности. У видео проверена страница/индексация, но не полное воспроизведение и содержание всей записи. Для SA Wiki доступна оболочка и поисковая индексация, поэтому справочник оставлен дополнительным. Полный реестр с типом и статусом каждого источника входит в комплект.

Код	Материал и ссылка	Язык	Проверка
R02	БА и СА в SA Wiki	RU	Оболочка и индексация подтверждены Полный текст не извлечен
R03	BPMN в SA Wiki	RU	Оболочка и индексация подтверждены Полный текст не извлечен
S01	Scrum Guide	EN	Открыта
S02	INVEST для пользовательских историй	EN	Открыта
S03	Gherkin Reference	EN	Открыта
S04	От хочу к ТЗ	RU	Открыта
S05	Работа с требованиями на разных этапах проекта	RU	Открыта
S06	Бизнес и системный аналитик Что нужно знать	RU	Открыта
S07	Критерии приемки для пользовательских историй	RU	Открыта
S08	Как сделать BPMN диаграмму чуточку лучше	RU	Открыта
S09	BPMN Reference	EN	Открыта
S10	BPMN Examples	EN	Открыта
S11	BPMN 2.0.2	EN	Открыта
S12	UML Sequence в PlantUML	RU	Открыта
S13	UML State в PlantUML	RU	Открыта
S14	UML Use Case в PlantUML	RU	Открыта
S15	UML 2.5.1	EN	Открыта
S16	Как работает Интернет	RU	Открыта
S17	Обзор протокола HTTP	RU	Открыта
S18	Коды состояния HTTP	RU	Открыта
S19	JSON	RU	Открыта
S20	Инструменты разработчика в браузере	EN	Открыта
S22	Язык SQL в документации PostgreSQL	RU	Открыта
S23	Ограничения данных в PostgreSQL	RU	Открыта

Код	Материал и ссылка	Язык	Проверка
S24	PostgreSQL Tutorial	EN	Открыта
S28	SQLBolt	EN	Открыта
S29	Интерактивный тренажер по SQL	RU	Подтверждена поиском Страница напрямую не загрузилась
S30	PostgreSQL Downloads	EN	Открыта
S31	DBeaver Community	EN	Открыта
S32	Postman Quick Start	EN	Открыта
S34	OpenAPI 3.1.1	EN	Открыта
S36	Swagger Editor	EN	Оболочка открыта Интерактивный редактор отдельно не проверен
S37	Creating your first JSON Schema	EN	Открыта
S38	Проектирование спецификации OpenAPI	RU	Открыта
S39	Как мы ломали и собирали API заново	RU	Открыта
S41	XML 1.0	EN	Открыта
S42	SOAP 1.2 Messaging Framework	EN	Открыта
S53	Webhooks	EN	Открыта
S54	RabbitMQ Tutorials	EN	Открыта
S61	OWASP API Security Top 10 2023	EN	Открыта с перенаправлением
S62	Authorization Cheat Sheet	EN	Открыта
S63	Logging Cheat Sheet	EN	Открыта
S64	Service Level Objectives	EN	Открыта
S67	Pro Git	RU	Открыта
S72	RFC 9457 Problem Details for HTTP APIs	EN	Открыта
S73	RFC 9110 HTTP Semantics	EN	Открыта
S74	Как мы унифицировали техническое интервью аналитика	RU	Открыта
S76	Forms Tutorial	EN	Открыта
S77	Kanban	EN	Открыта
V01	Рисуем схему с нуля BPMN Практика в diagrams net	RU	Подтверждено поиском Плеер не проверен
V02	Activity Diagram построение и применение	RU	Страница записи открыта Плеер не проверен
V03	CS50 SQL Week 0 Querying	EN	Страница лекции открыта Плеер не проверен
V04	CS50 SQL Week 1 Relating	EN	Страница лекции открыта Плеер не проверен
V05	CS50 SQL Week 2 Designing	EN	Страница лекции открыта Плеер не проверен
V07	Kafka 101 Events	EN	Страница урока открыта Плеер не проверен